

LECTURE – ACCESS MODIFIERS

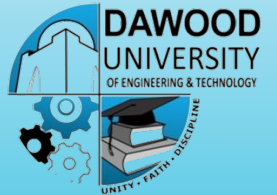
OOP (JAVA)



MR. MUHAMMAD SALEEM
LECTURER
DEPARTMENT OF COMPUTER SCIENCE

Dawood University of Engineering and Technology, Karachi

AGENDA



- Access modifiers in java
 - Public
 - Private
 - Protected
 - Default
- Examples

ACCESS MODIFIERS IN JAVA

- There are two types of modifiers in Java: **access modifiers** and **non-access modifiers**.
- The access modifiers in java specifies the accessibility of a class and its members.
- The access modifiers in Java specifies the accessibility or scope of a field, method, constructor, or class.
- We can change the access level of fields, constructors, methods, and class by applying the access modifier on it.

ACCESS MODIFIERS IN JAVA

- There are four types of Java access modifiers:
 1. **Private:** The access level of a private modifier is only within the class. It cannot be accessed from outside the class.
 2. **Default:** The access level of a default modifier is only within the package. It cannot be accessed from outside the package. If you do not specify any access level, it will be the default.
 3. **Protected:** The access level of a protected modifier is within the package and outside the package through child class. If you do not make the child class, it cannot be accessed from outside the package.
 4. **Public:** The access level of a public modifier is everywhere. It can be accessed from within the class, outside the class, within the package and outside the package.

UNDERSTANDING JAVA ACCESS MODIFIERS

Access Modifier	within class	within package	outside package by subclass only	outside package
Private	Y	N	N	N
Default	Y	Y	N	N
Protected	Y	Y	Y	N
Public	Y	Y	Y	Y

Package a

```
public class Test{  
    private int a;  
}
```

Package a

```
public class Test1{  
    protected int a;  
}
```

Package b

```
public class Test5 extends Test1{  
    Test1 t1 = new Test1();  
    SOP(t1.a);  
  
}
```

Package a

```
public class Test2{  
    int a;  
}
```

Package a

```
public class Test3{  
    public int a;  
}
```

Package b

```
public class Main{  
    public static void m(){  
        Test3 t3 = new Test3();  
        SOP(t3.a);  
    }  
}
```



Bean Classes
Accessor & Mutators
Setter & Getters
Read only & Write only methods

1) PRIVATE

- The private access modifier is accessible only within the class.
- **Simple example of private access modifier**
- In this example, we have created two classes A and Simple. A class contains private data member and private method. We are accessing these private members from outside the class, so there is a compile-time error.

EXAMPLE

```
class A{  
private int data=40;  
private void msg(){System.out.println("Hello java");}  
}
```

```
public class Simple{  
public static void main(String args[]){  
A obj=new A();  
System.out.println(obj.data);  
obj.msg();  
}
```

WHAT IF YOU MAKE ANY ROLE OF PRIVATE CONSTRUCTOR

- If you make any class constructor private, you cannot create the instance of that class from outside the class. For example:

2) DEFAULT

- If you don't use any modifier, it is treated as **default** by default.
- The default modifier is accessible only within package.
- It cannot be accessed from outside the package.
- It provides more accessibility than private.

EXAMPLE OF DEFAULT

```
package pack;  
class A{  
    void msg(){System.out.println("Hello");}  
}
```

```
package mypack;  
import pack.*;  
class B{  
    public static void main(String args[]){  
        A obj = new A();//Compile Time Error  
        obj.msg();//Compile Time Error  
    }  
}
```

In the above example, the scope of class A and its method msg() is default so it cannot be accessed from outside the package.

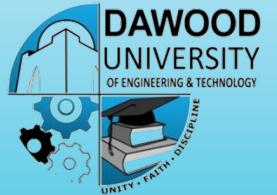
3) PROTECTED

- The **protected access modifier** is accessible within package and outside the package but through inheritance only.
- The protected access modifier can be applied on the data member, method and constructor.
- It can't be applied on the class.
- It provides more accessibility than the default modifier.

```
package pack;  
public class A{  
protected void msg(){System.out.println("Hello");}  
}
```

```
package mypack;  
import pack.*;  
  
class B extends A{  
    public static void main(String args[]){  
        B obj = new B();  
        obj.msg();  
    }  
}
```

ENCAPSULATION



- **Encapsulation in Java** is a *process of wrapping code and data together into a single unit*, for example, a capsule which is mixed of several medicines.
- We can create a fully encapsulated class in Java by making all the data members of the class private. Now we can use setter and getter methods to set and get the data in it.
- The **Java Bean** class is the example of a fully encapsulated class.

ADVANTAGE OF ENCAPSULATION IN JAVA

- By providing only a setter or getter method, you can make the class **read-only** or **write-only**. In other words, you can skip the getter or setter methods.
- It provides you the **control over the data**.
- Suppose you want to set the value of id which should be greater than 100 only, you can write the logic inside the setter method. You can write the logic not to store the negative numbers in the setter methods.
- It is a way to achieve **data hiding** in Java because other class will not be able to access the data through the private data members.

ACCESSOR AND MUTATOR EXAMPLE

```
package com.javatpoint;  
class Test{  
public static void main(String[] args  
) {  
    //creating instance of the encapsulat  
    ed class  
    Student s=new Student();  
    //setting value in the name member  
    s.setName("vijay");  
    //getting value of the name member  
    System.out.println(s.getName());  
}
```

READ ONLY METHOD

```
public class Student{  
    //private data member  
    private String college="AKG";  
    //getter method for college  
    public String getCollege(){  
        return college;  
    }  
}
```

```
s.setCollege("KITE");//will render compile time error
```

```
public class Student{  
    //private data member  
    private String college;  
    //getter method for college  
    public void setCollege(String college){  
        this.college=college;  
    }  
}
```

System.out.println(s.getCollege());*//Compile Time Error, because there is no such method*

System.out.println(s.college);*//Compile Time Error, because the college data member is private.*

//So, it can't be accessed from outside the class

Thanks 😊